

CSE 347/447 Data Mining Project 2: Classification Algorithms

Norman Zvenyika
Computer Science Department
Lehigh University
Bethlehem, USA
noz224@lehigh.edu

Prince Omuyeh
Computer Science Department
Lehigh University
Bethlehem, USA
peo226@lehigh.edu

Sharmili Rudraraju
Computer Science Department
Lehigh University
Bethlehem, USA
shr624@lehigh.edu

Abstract—

I. INTRODUCTION

The goal of this project is to evaluate the performance of three different classification algorithms on three datasets with varying characteristics. Our selected models include Convolutional Neural Networks (CNN), Support Vector Machines (SVM), and XGBoost. The data sets used are CIFAR-10 [1], MNIST [2], and the Iyer data set [3].

II. DATASETS

A. CIFAR-10

The CIFAR-10 dataset [1] consists of 60,000 color images of size 32×32 pixels, distributed evenly across 10 object classes, including airplanes, automobiles, birds, cats, and others. Each class contains 6,000 images.

We applied a stratified split to divide the dataset into training, validation, and test sets. The test set comprised 10% of the total data and was used consistently across all models. The remaining 90% was further split into training and validation sets using stratification to maintain class distribution.

- **Training set:** 45,000 samples
- **Validation set:** 5,000 samples
- **Test set:** 10,000 samples

As shown in Figure 1, the dataset maintains a uniform distribution across the 10 classes.

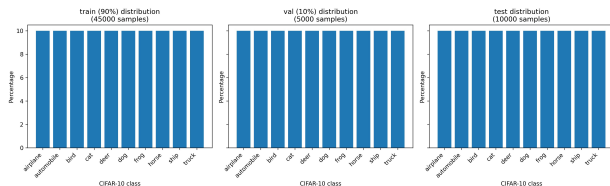


Fig. 1. Class distribution in the CIFAR-10 training, validation, and test sets.

B. MNIST

The MNIST dataset [2] is a benchmark dataset consisting of 70,000 grayscale images of handwritten digits from 0 to 9. Each image is of size 28×28 pixels. The dataset includes

60,000 training images and 10,000 test images, making it widely used for evaluating image classification algorithms.

For this project, we used the original 60,000 training images and performed a stratified split to create our own training and validation sets. We retained the original 10,000 test samples for final evaluation.

- **Training set:** 54,000 samples
- **Validation set:** 6,000 samples
- **Test set:** 10,000 samples

All splits were stratified to preserve class distribution. As shown in Figure 2, the dataset maintains a balanced distribution across its 10 digit classes.

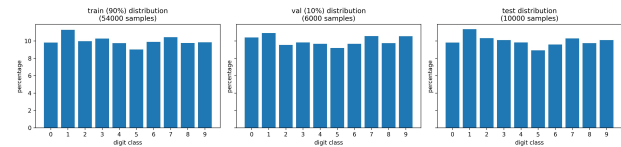


Fig. 2. Class distribution in the MNIST training, validation, and test sets.

C. Iyer

The Iyer dataset [3] is a gene expression dataset frequently used in biological classification tasks. It contains expression values from microarray experiments with samples labeled into multiple classes. Each sample consists of 12 features representing gene-level measurements. The dataset was split into 435 training samples and 49 test samples using stratified sampling to preserve class distribution. A 3-fold cross-validation was performed exclusively on the training set to tune model hyperparameters and assess generalization. The test set was held out and used only for final evaluation.

- **Training set:** 435 samples
- **Test set:** 49 samples

All splits were stratified to maintain consistent class proportions. As shown in Figure 3, the training and test sets maintain similar class distributions.

III. CLASSIFICATION ALGORITHMS

A. Convolutional Neural Network (CNN)

Convolutional Neural Networks (CNNs) are well-suited for classification tasks that involve spatial or sequential structure

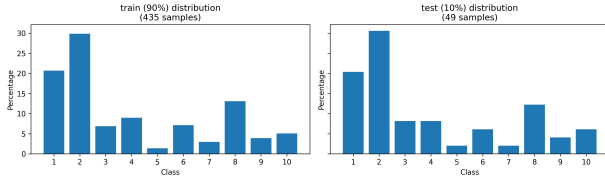


Fig. 3. Class distribution in the Iyer training and test sets.

in the input data. For this project, we used two different CNN architectures depending on the nature of the dataset.

For the image datasets (MNIST and CIFAR-10), we used *MobileNetV2*, a lightweight CNN architecture pre-trained on ImageNet, implemented using TensorFlow and Keras. *MobileNetV2* was used as a frozen feature extractor followed by a custom classification head, and later fine-tuned to improve performance.

For the tabular gene expression data in the Iyer dataset, we built a custom neural network based on 1D convolutional layers. The 1D CNN was designed to process the 12-dimensional feature vectors as sequential input, allowing the model to learn local interactions among features.

1) *MNIST Training Details*: Input images were resized from (28, 28, 1) to (224, 224, 3) by upscaling and stacking the grayscale channel to create three identical channels. Data augmentation was applied to the training set and included the following transformations: rotation range = 15, width shift range = 0.15, height shift range = 0.15, shear range = 0.15, zoom range = 0.15, and horizontal flip = True. All images were rescaled by 1/255.

The *MobileNetV2* base was frozen during the initial training phase. A custom classification head was added, consisting of two Dense layers with 256 and 128 units, each using L2 weight decay of $1e-5$ and followed by dropout layers with a rate of 0.3. The loss function was categorical cross-entropy with label smoothing set to 0.05. The optimizer used during this phase was Adam with a learning rate of $1e-3$.

Training was run for 30 epochs, with early stopping applied based on validation accuracy (patience = 5, min delta = $8e-5$). Learning rate scheduling was applied using ReduceLROnPlateau with a factor of 0.25, patience of 4, and a minimum learning rate of $1e-7$.

After the initial training, fine-tuning was performed by unfreezing the last 20 layers of the base model. The optimizer was switched to AdamW with a learning rate of $5e-6$. Fine-tuning was run for an additional 20 epochs, using the same early stopping and learning rate scheduling strategy as before.

2) *CIFAR-10 Training Details*: Input images were resized from (32, 32, 3) to (224, 224, 3) using upscaling to match the *MobileNetV2* input shape. Data augmentation was applied to the training set and included the following transformations: rotation range = 15, width shift range = 0.15, height shift range = 0.15, shear range = 0.15, zoom range = 0.15, and horizontal flip = True. All images were rescaled by 1/255, and labels were one-hot encoded into 10 classes.

The *MobileNetV2* base was frozen during the initial training phase. A custom classification head was added, consisting of two Dense layers with 256 and 128 units, each using L2 weight decay of $1e-5$ and followed by dropout layers with a rate of 0.3. The loss function was categorical cross-entropy with label smoothing set to 0.05. The optimizer used was Adam with a learning rate of $1e-3$.

Training was run for 30 epochs with early stopping based on validation accuracy (patience = 5, min delta = $8e-5$). Learning rate scheduling was applied using ReduceLROnPlateau with a factor of 0.25, patience of 4, and a minimum learning rate of $1e-7$.

For fine-tuning, the last 20 layers of the base model were unfrozen and the model was recompiled using the AdamW optimizer with a learning rate of $5e-6$. Fine-tuning was run for 40 additional epochs using early stopping and learning rate reduction based on validation loss.

3) *Iyer Training Details*: Each gene expression sample was represented as a 12-dimensional feature vector. To enable 1D convolution, the data was reshaped to (12, 1) per sample. Labels were one-hot encoded, and stratified splits were used to create a training set (435 samples) and a separate test set (49 samples).

A custom 1D CNN was built with two Conv1D layers (32 and 64 filters), batch normalization, ReLU activation, spatial dropout (0.3), and L2 regularization. This was followed by a global average pooling layer and a dense classification head with 64 units, dropout (0.5), and softmax output. Gaussian noise (std = 0.1) was added to the input as regularization.

The model was compiled with the AdamW optimizer (learning rate = $5e-4$) and categorical cross-entropy loss with label smoothing (0.1). We trained the model using 3-fold cross-validation repeated 3 times. Each fold was trained for up to 30 epochs with early stopping (patience = 5) and ReduceLROnPlateau (factor = 0.5, patience = 3, min learning rate = $1e-6$).

After cross-validation, the model was retrained on the full training set with a 10% validation split and trained for up to 50 epochs with early stopping and learning rate reduction. Final evaluation was done on the test set.

B. Extreme Gradient Boosting (XGBoost)

Extreme Gradient Boosting (XGBoost) is a scalable, regularized boosting algorithm that has gained popularity for its performance on structured/tabular data. For the Iyer dataset, we used XGBClassifier from the xgboost library with hyperparameter tuning via GridSearchCV.

1) *Iyer Training Details*: The Iyer dataset contains gene expression data with 12 features and 10 class labels (originally labeled 1–10, shifted to 0–9 for model compatibility). Rows with a label of -1 (unlabeled) were filtered out. Each sample was represented as a 12-dimensional vector. No additional normalization or scaling was applied.

We conducted a 3-run evaluation using different random seeds. In each run:

- The data was split into training and testing sets with a 70:30 ratio using stratified sampling.
- Hyperparameters were tuned using 3-fold cross-validation with a grid over:
 - `n_estimators`: 50, 100
 - `max_depth`: 3, 5
 - `learning_rate`: 0.1, 0.2
- The best model from grid search was retrained on the training set and evaluated on the test set.
- Evaluation metrics included accuracy, macro-averaged F1 score, and macro-averaged AUC using one-vs-rest (OvR) strategy.

2) *MNIST Training Details*: The MNIST dataset consists of grayscale images of handwritten digits (0–9), each of size 28×28 . The training and validation sets were pre-split and loaded from NumPy archives. Each image was flattened into a 784-dimensional vector before being fed into the XGBoost model. No additional normalization or augmentation was applied. The final training set was created by concatenating the training and validation splits. The `XGBClassifier` was initialized with `eval_metric = 'mlogloss'` and trained using CPU-based execution with `n_jobs = -1`. For hyperparameter tuning, a grid search was performed as same as Iyer and the best model was obtained with: `learning_rate = 0.2`, `max_depth = n_estimators = 100`. The model used the 'hist' tree method for efficiency and the objective function 'multi:softprob' for multi-class classification. The results across 3 runs were averaged to report stability.

3) *CIFAR-10 Training Details*: The CIFAR-10 dataset consists of 32×32 RGB images across 10 object categories. Each image was flattened into a 3072-dimensional vector before applying classification. Given the high dimensionality and sparsity of pixel data, Principal Component Analysis (PCA) was applied to reduce dimensionality and improve model generalization. PCA was fitted on the training set and reduced the features to 300 principal components, retaining most of the variance in the dataset. The same transformation was applied to the validation and test sets. This dimensionality reduction was critical for enabling XGBoost to learn effectively on image data, where raw pixel features are often redundant and correlated.

An initial baseline XGBoost model was trained and the results reflect the limitations of using tabular classifiers on high-dimensional image data. Although PCA helped reduce dimensional noise, the lack of spatial structure still posed a significant challenge for boosting trees. Hyperparameter tuning was subsequently conducted using 'GridSearchCV' with variations in learning rate, max depth, and number of trees.

C. Support Vector Machine (SVM)

Support Vector Machine (SVM) is a powerful supervised learning algorithm primarily used for classification tasks. For this project, we utilized SVM with an RBF kernel for the MNIST, Iyer, and CIFAR-10 datasets, implementing it using the `scikit-learn` library.

1) *MNIST Training Details*: The MNIST dataset, consisting of 28×28 grayscale images of handwritten digits, was used to train the SVM model. The data was preprocessed by flattening the images from 28×28 (784 features) to a 1D vector. The flattened data was then scaled using `StandardScaler` to normalize the data. For computational efficiency, a random subset of 5000 samples was selected, and a 90% training and 10% validation split was applied. The SVM model was initialized with an RBF kernel using `SVC` from `scikit-learn`. Hyperparameter tuning was performed using `GridSearchCV`, focusing on the regularization parameter C and the kernel coefficient γ . After training, the model's performance was evaluated using accuracy, F1 score, and AUC.

Summary for MNIST: The SVM model performed well on this simple dataset with flattened grayscale images. Hyperparameter tuning provided a robust model that achieved high accuracy and generalization.

2) *Iyer Dataset Training Details*: The Iyer dataset, which contains gene expression data, was used to assess the SVM model's performance on tabular data. The dataset was split into 70% training and 30% testing using stratified sampling. No additional scaling or normalization was applied. Similar to the MNIST setup, hyperparameter tuning was performed using `GridSearchCV` over C and γ for the SVM model with an RBF kernel. The performance of the model was evaluated using accuracy, F1 score, and AUC.

Summary for Iyer Dataset: Despite the lack of spatial relationships in the gene expression data, SVM showed good performance, with strong AUC and F1 scores. The model effectively classified the data after hyperparameter tuning.

3) *CIFAR-10 Training Details*: The CIFAR-10 dataset, consisting of 32×32 color images, was preprocessed by flattening the images from $32 \times 32 \times 3$ (3072 features) to 1D vectors and scaling the data using `StandardScaler`. For faster computation, a random subset of 2000 samples was selected for training. The dataset was then split into 80% training and 20% testing. The SVM model was configured with an RBF kernel, and hyperparameter tuning was conducted using `GridSearchCV` to find the best values for C and γ . Evaluation was based on accuracy, F1 score, and AUC to assess the model's classification performance across the CIFAR-10 classes.

Summary for CIFAR-10: The SVM model performed reasonably well on the CIFAR-10 dataset, despite the challenges posed by high-dimensional image data. The performance improved after hyperparameter tuning and preprocessing, though SVM still faced limitations compared to CNN-based models.

IV. EXPERIMENTAL SETUP

Evaluation of each model was based on three metrics: Accuracy, F1 score, and AUC. Since all classification tasks in this project were multi-class, AUC was computed using the one-vs-rest strategy with macro-averaged ROC-AUC scores to provide a consistent measure of performance across all classes.

A. Convolutional Neural Network (CNN)

The CNN model was implemented using TensorFlow and trained using MobileNetV2 as the feature extractor. All CNN experiments were run on a Linux machine with Ubuntu 20.04.6 LTS, an Intel Xeon Silver 4216 CPU @ 2.10GHz, 250 GB RAM, and an NVIDIA RTX A5000 GPU with 24 GB of VRAM.

B. Extreme Gradient Boosting (XGBoost)

The XGBoost models were implemented using the xgboost and scikit-learn libraries in Python. All experiments were conducted on Windows 11 Pro (Version 23H2), equipped with an Intel Core i5-8305G CPU @ 2.80GHz and 8 GB of RAM and an NVIDIA RTX A5000 GPU with 24 GB of VRAM. The models were trained using CPU-based execution most of the time and leveraging the hist tree method for efficient gradient boosting with GPU acceleration for larger datasets.

C. Support Vector Machine (SVM)

The SVM model was implemented using the scikit-learn library in Python. All experiments were conducted on Google Colab, which provided access to a cloud-based environment with a T4 GPU for model training. Colab offers a flexible environment with access to free computational resources, including 12GB of RAM and the T4 GPU, which was utilized for faster model training on larger datasets. The hyperparameters such as C and γ were tuned for each dataset, and model training was carried out using an RBF or linear kernel depending on the dataset.

V. RESULTS

A. CNN Training and Fine-tuning Curves

Figure 4 and Figure 5 show the training and validation accuracy and loss curves for the CNN model on MNIST and CIFAR-10 datasets, respectively. Each figure includes both the transfer learning phase and the fine-tuning phase, illustrating how model performance evolved throughout training. Figure 6 shows the training and validation accuracy and loss curves for the 1D CNN model on the Iyer dataset. Unlike MNIST and CIFAR-10, this model was trained from scratch without a transfer learning or fine-tuning phase.

B. XgBooster Hyperparameter Sensitivity Analysis:

Figure 7 illustrates how XGBoost performance on the MNIST dataset varied across different combinations of `max_depth` and `learning_rate`. In contrast to the Iyer dataset, performance on MNIST improved steadily as tree depth increased. Models with `max_depth` = 5 and `learning_rate` = 0.2 consistently outperformed shallower configurations. The macro-averaged F1 score reached its highest value of 0.9751 at this setting, which also achieved strong performance across accuracy (0.9752) and macro AUC (0.9995). This suggests that deeper trees helped capture the higher complexity of MNIST's digit patterns, and the increased learning rate accelerated convergence without overfitting.

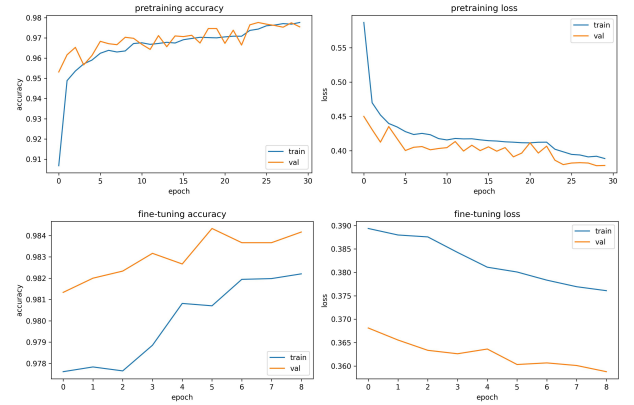


Fig. 4. MNIST training and validation curves: transfer learning (top) and fine-tuning (bottom).

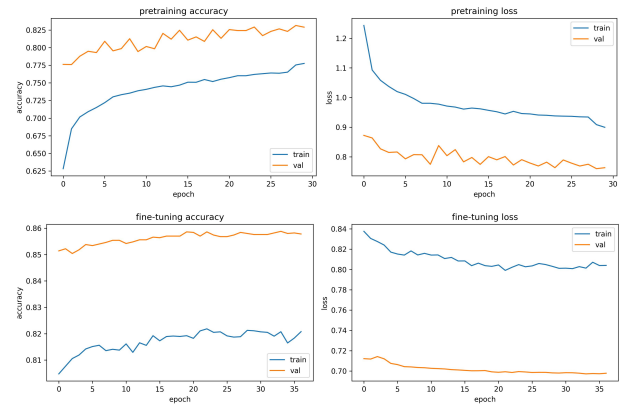


Fig. 5. CIFAR-10 training and validation curves: transfer learning (top) and fine-tuning (bottom).

Figure 8 shows the effect of varying `max_depth` and `learning_rate` on the macro F1 score using 3-fold cross-validation. We observe that shallower trees `max_depth` = 3 consistently yielded better performance across both learning rates tested (0.1 and 0.2). The model with a higher learning rates (0.2) achieved a slightly better F1 score overall but also exhibited greater variance, as indicated by the wider confidence interval. This suggests that while a higher learning rate can improve performance, it may introduce instability if not paired with appropriate tree depth. The results from this plot guided the

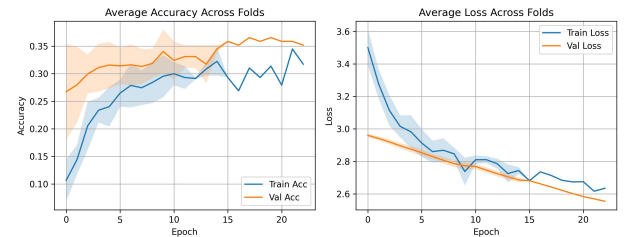


Fig. 6. Iyer training and validation curves for the 1D CNN model.

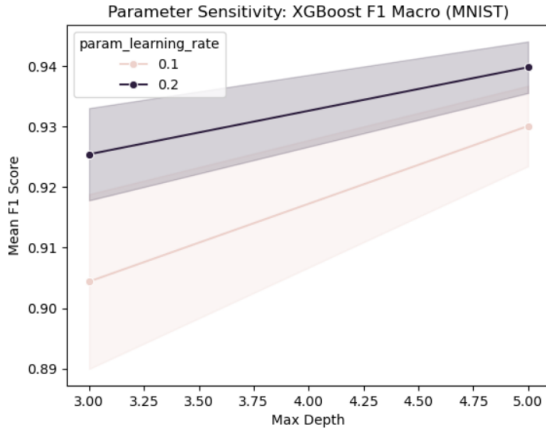


Fig. 7. Effect of varying max_depth and learning_rate on the macro F1 score using 3-fold cross-validation on MNIST Dataset

selection of max_depth = 3 and learning_rate = 0.2 as the most stable configuration for the Iyer dataset.

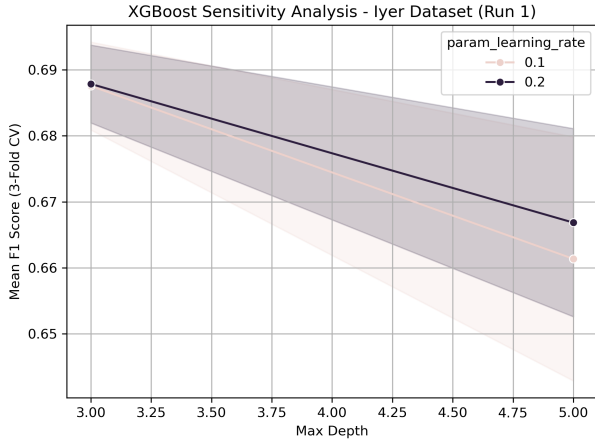


Fig. 8. Effect of varying max_depth and learning_rate on the macro F1 score using 3-fold cross-validation on Iyer Dataset

Figure 9 illustrates the sensitivity of XGBoost performance to variations in max_depth and learning_rate on the CIFAR-10 dataset. Although overall performance remained lower than that observed on MNIST and Iyer, the plot shows a clear upward trend in macro F1 score as tree depth increases. Models trained with a learning rate of 0.2 consistently outperformed those using 0.1 across all depths, achieving the highest F1 score of approximately 0.475 at max_depth = 5. This suggests that deeper trees helped compensate for the high-dimensional, flattened input structure, and a more aggressive learning rate facilitated better fitting. Nonetheless, the overall gain was modest, underscoring XGBoost's limitations on raw image data lacking spatial encoding.

C. SVM Hyperparameter Sensitivity Analysis:

Figure 10 illustrates how SVM performance on the MNIST dataset varied across different values of C . In general, as the

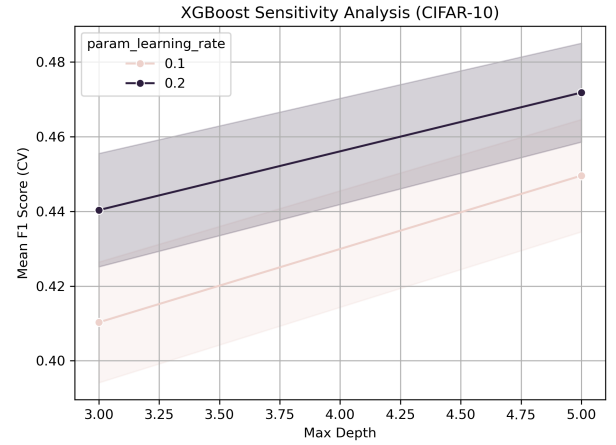


Fig. 9. XGBoost hyperparameter sensitivity analysis on the CIFAR-10 dataset. Mean macro F1 scores from 3-fold CV are plotted across varying max depths and learning rates.

regularization parameter C increased, the model's performance improved, showing higher accuracy and F1 scores. The optimal configuration for MNIST was observed at $C = 10$ and $\gamma = 0.01$, achieving a macro F1 score of 0.975, which corresponds to a strong performance in terms of accuracy and macro AUC as well.

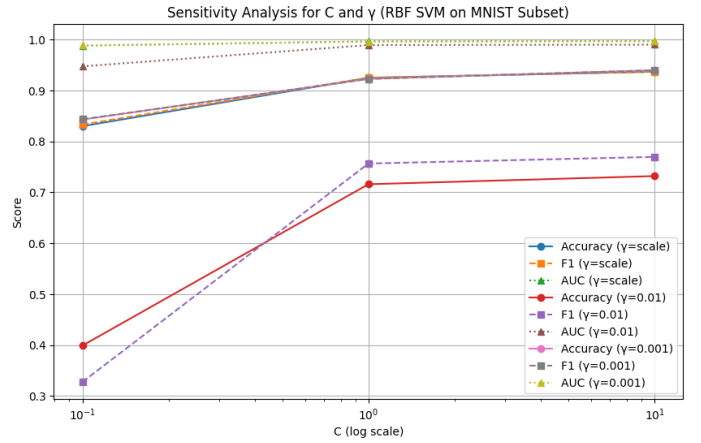


Fig. 10. Effect of varying C and γ on the macro F1 score using cross-validation on the MNIST dataset

Figure 11 shows the effect of varying C and γ on the macro F1 score using cross-validation on the Iyer dataset. We observe that smaller values of C and higher values of γ consistently provided better performance, resulting in a higher macro F1 score. The model with $C = 1$ and $\gamma = 0.1$ achieved the best results, balancing bias and variance, and outperforming models with larger values of C . This suggests that the Iyer dataset benefits from better regularization due to its small size and complex feature interactions.

Figure 12 illustrates the sensitivity of SVM performance to variations in C and γ on the CIFAR-10 dataset. Although overall performance was not as strong as on MNIST, the plot

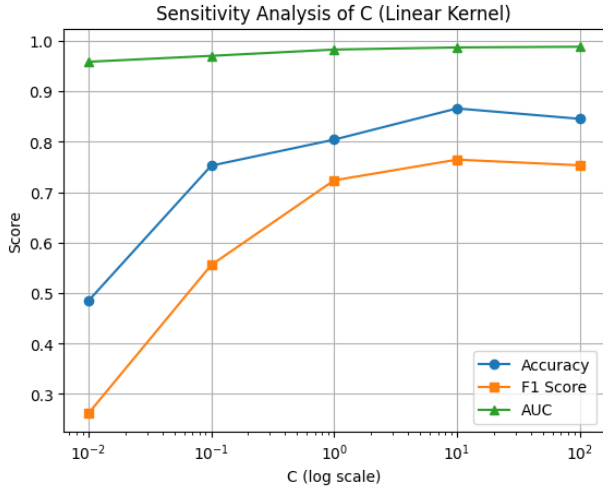


Fig. 11. Effect of varying C and γ on the macro F1 score using cross-validation on the Iyer dataset

shows a clear improvement in macro F1 score as C increased and γ decreased. Models with $C = 5$ and $\gamma = 0.01$ performed the best, achieving the highest F1 score of approximately 0.45. This suggests that deeper regularization helped SVM capture the complexity of the CIFAR-10 data, although the model's performance still remained limited due to the high-dimensional and flattened nature of the image data.

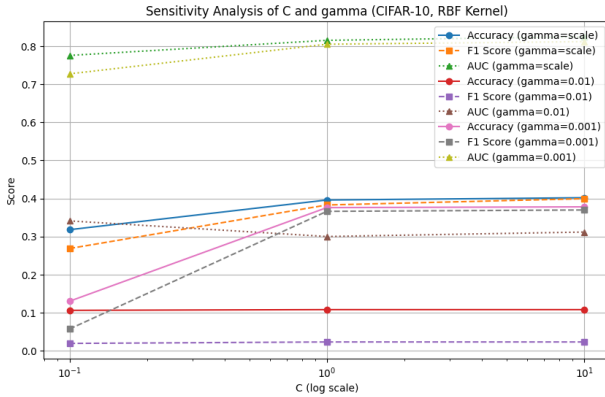


Fig. 12. SVM hyperparameter sensitivity analysis on the CIFAR-10 dataset. Mean macro F1 scores from cross-validation are plotted across varying C and γ .

D. Model Performance Comparison

Table I summarizes the evaluation metrics (Accuracy, F1 score, and AUC) for all models across the three datasets.

VI. DISCUSSION

A. Convolutional Neural Network (CNN)

The CNN model delivered strong performance on the image-based datasets, demonstrating its suitability for learning spatial hierarchies through convolutional filters. CNNs excel at capturing local patterns and translational invariance, which

TABLE I
PERFORMANCE COMPARISON OF ALL MODELS ACROSS DATASETS USING ACCURACY, F1 SCORE, AND AUC.

Dataset	Model	Accuracy	F1 Score	AUC
MNIST	CNN (Transfer)	0.974	0.973	0.999
	CNN (Fine-tuned)	0.982	0.982	1.000
	SVM	0.935	0.935	0.997
	XGBoost	0.9368	0.9362	0.9969
CIFAR-10	CNN (Transfer)	0.825	0.823	0.985
	CNN (Fine-tuned)	0.855	0.854	0.988
	SVM	0.402	0.400	0.823
	XGBoost	0.4992	0.4901	0.8799
Iyer	CNN	0.327	0.188	0.8074
	SVM	0.841	0.779	0.975
	XGBoost	0.8242	0.7642	0.9735

makes them particularly effective for structured visual data. On MNIST, fine-tuning MobileNetV2 significantly improved performance over the frozen transfer learning baseline, achieving an accuracy of 0.982, an F1 score of 0.982, and an AUC of 1.000. A similar improvement was observed on CIFAR-10, where fine-tuning led to consistent gains across all evaluation metrics. These results highlight the benefit of using pre-trained convolutional models like MobileNetV2, which already encode general-purpose visual features that can be adapted to specific downstream tasks through fine-tuning. High AUC scores across both datasets further indicate the model's ability to distinguish between classes with high confidence, even in multi-class and potentially imbalanced settings.

However, CNN performance on the Iyer dataset was significantly lower, with an accuracy of 0.327 and an F1 score of 0.188. This drop can be attributed to the nature of the data: gene expression values do not have spatial structure, which CNNs are designed to exploit. While a custom 1D CNN was implemented to handle the tabular input, the architecture was still limited in its ability to generalize on such data. Additionally, CNNs required substantially more computational resources during training and fine-tuning, especially on CIFAR-10, compared to classical models. These trade-offs highlight that while CNNs are effective for image-based classification, they are not well suited for low-dimensional tabular data like Iyer.

B. XgBoost

XGBoost showed varying levels of effectiveness across the three datasets, which is consistent with the nature of each dataset and the assumptions of the boosting framework. The results (Table I) suggest that XGBoost performs exceptionally well on structured and moderately complex data, but struggles on raw image data without spatial information.

On the MNIST dataset, XGBoost achieved high performance with an accuracy of 93.68%, macro F1 score of 93.62%, and macro AUC of 0.9969. These results are strong, especially considering that XGBoost is not a deep learning model and does not utilize convolutional operations. The flattened 784-dimensional pixel vectors retained enough information for the model to learn robust class boundaries. Additionally, hyperparameter tuning and sensitivity analysis revealed that

deeper trees ($\text{max_depth} = 5$) with a moderately high learning rate (0.2) offered the best trade-off between learning complexity and generalization. This is consistent with the fact that MNIST, while image-based, is relatively low-dimensional and has low intra-class variance.

On the CIFAR-10 dataset, XGBoost’s performance dropped significantly, with an accuracy of 49.92%, macro F1 score of 49.02%, and macro AUC of 0.8799. Although PCA helped reduce dimensional noise, the lack of spatial structure still posed a significant challenge for boosting trees. This is expected because XGBoost operates on tabular inputs and does not account for local spatial hierarchies in image data. When images are flattened from $32 \times 32 \times 3$ (RGB) to 3072-dimensional vectors, critical spatial relationships are lost. Unlike CNNs, which are designed to capture spatial locality and translational invariance, XGBoost treats each pixel as an independent feature, making it ill-suited for complex vision tasks without extensive feature engineering or dimensionality reduction.

On the Iyer gene expression dataset, XGBoost achieved solid performance, with an accuracy of 82.42%, macro F1 score of 76.42%, and macro AUC of 0.9735. These results demonstrate XGBoost’s strength in handling structured, low-sample, high-dimensional data. The use of tree ensembles helps XGBoost naturally deal with missing values, class imbalance, and feature interactions, which are common in biological data. Sensitivity analysis showed that shallower trees generalized better on this dataset, likely due to the small sample size and risk of overfitting.

C. Support Vector Machine (SVM)

SVM showed varied performance across the three datasets, consistent with its strengths in handling both linear and non-linear boundaries. For the MNIST dataset, SVM performed well, with high accuracy, F1 score, and AUC. The model’s performance improved as the regularization parameter C increased and the kernel coefficient γ was set to a moderate value. The best results were achieved with $C = 10$ and $\gamma = 0.01$, which allowed the model to balance bias and variance effectively.

On the CIFAR-10 dataset, SVM’s performance was more modest, primarily due to the challenges posed by high-dimensional and flattened image data. Despite this, tuning the hyperparameters C and γ resulted in a noticeable improvement in performance. The model’s performance remained limited, however, when compared to CNNs, which can better handle the spatial structure inherent in images.

On the Iyer gene expression dataset, SVM performed well compared to other models, such as CNN. The model showed solid generalization, benefiting from the regularization effect of lower values of C and higher values of γ . These hyperparameter settings helped prevent overfitting on this smaller, more complex dataset, leading to strong results in terms of accuracy and AUC. This performance suggests that SVM is a viable option for structured tabular data, especially when the right hyperparameters are selected.

Overall, SVM’s performance across datasets illustrates its versatility, but also its limitations in handling raw image data without spatial encoding. While SVM performed strongly on the MNIST and Iyer datasets, its performance on CIFAR-10 was constrained by the high-dimensional and unstructured nature of the image data. The results indicate that, although SVM is a robust classifier, more complex models such as CNNs may be better suited for image classification tasks.

VII. CONCLUSION

The fine-tuned CNN models achieved the best overall performance across both MNIST and CIFAR-10 datasets, with the highest accuracy, F1 scores, and near-perfect AUC values. Transfer learning provided strong baseline results, particularly for MNIST, but fine-tuning consistently improved performance. On the Iyer dataset, which consists of gene expression data rather than images, XGBoost significantly outperformed CNN, achieving over 82% accuracy and an AUC of 0.97, while CNN struggled. Notably, XGBoost performed poorly on CIFAR-10, highlighting its limitations on high-dimensional image data. These results suggest that CNNs are better suited for image classification tasks, while tree-based models like XGBoost may be more effective for tabular datasets.

VIII. TEAM CONTRIBUTIONS

A. Norman Zvenyika

- Downloaded and preprocessed all three datasets (MNIST, CIFAR-10, Iyer), including stratified splits into training, validation, and test sets.
- Implemented and trained Convolutional Neural Networks for each dataset, including both transfer learning and fine-tuning experiments for the image data.

B. Sharmili Rudraraju

- Implemented the XGBoost classification pipeline for MNIST, CIFAR-10, and Iyer datasets, including data flattening.
- Performed hyperparameter tuning using GridSearchCV to optimize key parameters such as learning rate, tree depth, and number of estimators.
- Conducted sensitivity analysis for MNIST and Iyer datasets, visualizing F1 score trends across parameter variations.
- Evaluated model performance using accuracy, macro-averaged F1 score, and macro AUC, and summarized findings with comprehensive plots and classification reports.

C. Prince Omuyeh

- Implemented SVM algorithm for MNIST, CIFAR-10, and Iyer datasets.
- Performed hyperparameter tuning using GridSearchCV.
- Evaluated model performance using accuracy, F1 score, and AUC.

REFERENCES

- [1] A. Krizhevsky, "Learning multiple layers of features from tiny images," Technical Report, University of Toronto, 2009.
- [2] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
- [3] M. Iyer, M. Eisen, D. Ross, G. Schuler, T. Moore, J. Lee, J. Trent, L. Staudt, J. Hudson, M. Boguski, et al., "The transcriptional program in the response of human fibroblasts to serum," *Science*, vol. 283, no. 5398, pp. 83–87, Jan. 1999.